tagd Math

Isaac N Colley

April 2026

1 Introduction

This document describes the mathematical structures already present in the `tagd` semantic-relational system and the TAGL language specification. Its purpose is not to invent a new mathematical model, but to state, as precisely as possible, the mathematics implicit in the current source and specification.

The central claim is that a tagspace is not merely a collection of labels. It has mathematical structure. In particular, a tagspace exhibits at least the following:

- a set of tags
- a rooted tree induced by subordinate relations
- a rank-based containership relation
- a partial order induced by rank-prefix containment
- additional labeled relations on nodes through predicates

These structures support the broader thesis that in `tagd`, information is structural: meaning is determined not only by nominal labels, but by position in the hierarchy and by relations attached to that position.

2 Preliminaries

2.1 Tags as a Set

Definition 2.1. *Let T be the set of all tags in a tagspace.*

Each tag has two forms of identity:

1. **Nominal identity**: a globally unique UTF-8 text label.
2. **Structural identity**: its subordinate relation and its predicate list.

Example TAGL statements:

```
>> mammal _type_of animal
```

```
>> dog _is_a mammal
```

Here, `dog`, `mammal`, and `animal` are all elements of T.

2.2 Subordinate Relations

Definition 2.2. *A subordinate relation is a binary relation on tags:*

$$_sub \subseteq T \times T$$

where $(a, b) \in _sub$ *means that tag* a *is subordinate to tag* b*.*

In TAGL surface syntax, subordinate relations may be written using particular sub-relators such as `_sub`, `_is_a`, or `_type_of`. The general structural role is the same: a subject is placed under a super-object in the tagspace hierarchy.

Example:

```
>> dog _is_a mammal
```

3 Tagspace as a Rooted Tree

3.1 Root

Definition 3.1. *There exists a distinguished root tag*

$$_entity \in T$$

which is axiomatic.

The root relation is:

```
_entity _sub _entity
```

This makes `_entity` the unique root of the tagspace.

3.2 Parent Function

Because each tag has exactly one subordinate relation, each non-root tag has exactly one parent in the hierarchy.

Definition 3.2. *Define the parent function*

$$\text{parent} : T \to T$$

such that

$$\text{parent}(a) = b$$

whenever a *is subordinate to* b*.*

Proposition 3.1. *The subordinate structure of a tagspace forms a rooted tree.*

Proof. Each tag has exactly one subordinate relation, and all tags ultimately descend from the distinguished root *_entity*. Therefore the hierarchy has a unique root and unique parent assignment for each node, which is exactly the defining shape of a rooted tree. □

3.3 Example

```
>> physical_object -^ _entity
>> living_thing _type_of physical_object
>> animal _type_of living_thing
>> mammal _type_of animal
>> dog _is_a mammal
```

This yields the tree fragment

$$_entity \leftarrow physical_object \leftarrow living_thing \leftarrow animal \leftarrow mammal \leftarrow dog$$

where arrows point from child to parent.

4 Rank and Containership

4.1 Rank Function

In `tagd`, each tag has a rank. A rank is a UTF-8 byte string whose prefixes encode containership in the tree.

Definition 4.1. *Define a rank function*

$$\mathrm{rank} : T \to \Sigma^*$$

where Σ^* is the set of finite UTF-8 strings.

The intended semantics are:

- the rank of a parent is a prefix of the rank of each descendant
- rank substrings define subtrees
- lexicographic collation of rank strings preserves hierarchy

4.2 Containership Relation

Definition 4.2. *Define containership on tags by*

$$b \preceq a \quad \Longleftrightarrow \quad \mathrm{rank}(b) \textit{ is a prefix of } \mathrm{rank}(a)$$

This means that b contains a in the hierarchical sense: a lies in the subtree rooted at b.

Remark 4.1. *This is the mathematical form of the source-level intuition: a parent rank prefixes all ranks in its subtree.*

4.3 Example

Suppose the ranks are:

$$\mathrm{rank}(_entity) = \epsilon, \quad \mathrm{rank}(mammal) = 1.2.4, \quad \mathrm{rank}(dog) = 1.2.4.7$$

Then $\mathrm{rank}(mammal)$ is a prefix of $\mathrm{rank}(dog)$, so $mammal \preceq dog$.

5 Tagspace as a Partially Ordered Set

5.1 Specialization Order

There are two equivalent ways to orient the order. For language design, the most useful one is usually the specialization order.

Definition 5.1. *Define*

$$a \leq b \quad \Longleftrightarrow \quad \mathrm{rank}(b) \textit{ is a prefix of } \mathrm{rank}(a)$$

Interpretation:

- $a \leq b$ means a is subordinate to b
- a is more specific than b
- b is more general than a

Proposition 5.1. *$(T, \leq)$ is a partially ordered set.*

Proof. **Reflexive:** Every rank is a prefix of itself, so $a \leq a$.

Transitive: If $\mathrm{rank}(b)$ is a prefix of $\mathrm{rank}(a)$ and $\mathrm{rank}(c)$ is a prefix of $\mathrm{rank}(b)$, then $\mathrm{rank}(c)$ is a prefix of $\mathrm{rank}(a)$, so $a \leq c$.

Antisymmetric: If $a \leq b$ and $b \leq a$, then each rank is a prefix of the other, hence the ranks are equal. In a tagspace, equal structural position implies the same node in the tree, so $a = b$. □

5.2 Interpretation

This partial order is not added from outside. It is already present in the system through rank-prefix containment. It formalizes the intuition that the tagspace expresses a hierarchy of specialization and generalization.

6 Subtrees and Set-Theoretic Structure

6.1 Subtrees as Sets

Given any tag $x \in T$, define its subtree by

$$\text{Subtree}(x) = \{y \in T \mid x \preceq y\}$$

This is the set of all descendants of x, including x itself.

6.2 Examples

If the tagspace contains:

```
>> animal _type_of living_thing
>> mammal _type_of animal
>> bird _type_of animal
>> dog _is_a mammal
>> cat _is_a mammal
```

then

$$\text{Subtree}(animal) = \{animal, mammal, bird, dog, cat, \dots\}$$

and

$$\text{Subtree}(mammal) = \{mammal, dog, cat, \dots\}$$

6.3 Intersection Behavior

For two tags $x, y \in T$, the intersection

$$\text{Subtree}(x) \cap \text{Subtree}(y)$$

is either:

- empty, if the subtrees are disjoint
- Subtree(x), if x lies below y
- Subtree(y), if y lies below x

This follows from the fact that a rooted tree has no cycles and each node has a unique path to the root.

7 A Topological View of Tagspaces

7.1 Motivation

The tagspace is already:

- a set T
- a rooted tree defined by subordinate relations

- a partially ordered set under rank-prefix containment

This naturally induces a topology. The goal of this section is to define that topology precisely and consistently.

7.2 Specialization Order (Canonical Choice)

We define the order in terms of specialization:

Definition 7.1. *For $a, b \in T$,*

$$a \leq b \quad \iff \quad \mathrm{rank}(b) \textit{ is a prefix of } \mathrm{rank}(a)$$

Interpretation:

- $a \leq b$ means a is more specific than b
- b is more general than a
- moving upward in the order moves toward generalization

Example:

```
>> dog _is_a mammal
>> mammal _type_of animal
```

$$dog \leq mammal \leq animal$$

7.3 Alexandrov Topology

Definition 7.2. *A subset $U \subseteq T$ is open if:*

$$x \in U \textit{ and } x \leq y \implies y \in U$$

That is, U is upward-closed with respect to the specialization order.

7.4 Interpretation

Because $x \leq y$ means "x is more specific than y":

- if a tag is in U, all more general tags are also in U
- open sets propagate upward toward generalization

Example:
If $dog \in U$, then:

$$mammal, animal, _entity \in U$$

7.5 Dual View (Subtree Interpretation)

While the topology is defined using the specialization order, it can be understood more intuitively in terms of the tree structure.

Define the subtree of a tag:

$$\mathrm{Subtree}(x) = \{y \in T \mid x \preceq y\}$$

where $x \preceq y$ means x is an ancestor of y.
These subtrees correspond to downward-closed sets in the tree.

Remark 7.1. *The Alexandrov topology defined above is the dual of the subtree containment structure. Both views arise from the same rank-prefix relation.*

7.6 Topological Properties

The topology satisfies:

- arbitrary unions of open sets are open
- finite intersections of open sets are open
- $\varnothing$ and T are open

Therefore, it defines a valid topology on T.

7.7 Structural Meaning

The topology encodes generalization and specialization:

- moving upward in the order corresponds to generalization
- open sets capture closure under generalization

Thus, the topology is not an external addition. It is a direct mathematical expression of the structure already present in the tagspace.

8 Hard Tags as a Universal Invariant Substructure

8.1 Hard Tags

Let

$$H \subseteq T$$

be the set of hard tags.

These are the tags defined axiomatically by the system, including tags such as:

- `_entity`
- `_sub`
- `_rel`
- `_number`
- `_interrogator`

8.2 Shared Base

Every tagspace contains the same hard tag base. Therefore H is a common invariant substructure across all tagspaces.

Remark 8.1. *The hard tag base is the structural common ground that makes all tagspaces mutually legible. Any extension of a tagspace is built on top of the same axiomatic base.*

8.3 Example

A bootstrap dump shows a fixed base such as:

```
_entity
_sub
_is_a
_type_of
_rel
_has
_can
_number
_integer
_float
_interrogator
_what
_search
```

No matter how different two tagspaces become, this hard-tag substructure remains shared.

9 Predicate Relations as Additional Structure

9.1 Predicates

A predicate relation is not part of the vertical containment tree. Instead, it adds labeled relational structure to nodes.

Definition 9.1. *A predicate is a relation of the form*

$$(r, O)$$

where r is a relator and O is an object list, applied to a subject.

A predicate with modifier may be represented more explicitly as

$$(r, o, m)$$

where r is the relator, o the object, and m the modifier.

9.2 Example

```
>> dog _has legs = 4
>> dog _can bark
```

These do not change the position of `dog` in the tree. They add horizontal relational structure to the node `dog`.

9.3 Mathematical View

A tagspace therefore has at least two interacting layers:

1. a vertical tree / order / topology given by subordinate relations
2. a horizontal labeled relational layer given by predicates

This is why the system is properly called semantic-relational. The meaning of a tag is not exhausted by its nominal name; it is expressed by both hierarchical placement and attached relations.

10 Structural Identity

10.1 Two Forms of Identity

A tag has:

1. nominal identity
2. structural identity

Nominal identity is its globally unique UTF-8 label.
Structural identity is determined by:

- its subordinate relation
- its rank
- its predicate list

10.2 Interpretation

This is a structuralist view of information. A tag is not meaningful in isolation. Its meaning is determined by where it sits in the tagspace and how it relates to other tags.

A concise formula is:

$$\text{meaning}(x) = \text{position}(x) + \text{relations}(x)$$

where "position" is given by subordinate structure and rank, and "relations" are given by predicates.

11 Canonical Representation

11.1 Text as Structural Serialization

A tagspace has a canonical textual representation through `dump()`.

This means that the full structure of the tagspace is representable in deterministic TAGL text.

11.2 Interpretation

A tagspace is not merely stored as text. Rather, the text is a canonical serialization of the underlying mathematical structure.

This is important because it allows:

- deterministic inspection
- canonical comparison
- hashable structural snapshots
- faithful reload of the same structure

12 Conclusion

The `tagd` system already contains a rich mathematical foundation.

A tagspace may be described as:

- a set of tags
- a rooted tree

- a rank-based containership hierarchy
- a partially ordered set under specialization
- an induced Alexandrov topology
- an invariant hard-tag substructure
- a relational layer of predicates attached to nodes

These are not external metaphors. They are mathematical descriptions of structure already present in the system.

This supports the broader thesis that in `tagd` and TAGL, information is structural.

13 Future Work

Natural continuations of this document include:

- formal treatment of query semantics as set and order operations
- formal treatment of predicate algebra
- morphisms between tagspaces
- merge operations as structure-preserving transformations
- closure, inheritance, and specialization in the topological model